

DOSSIER DE SÉCURITÉ APPLICATIVE

JurislA

Cartographie complète des risques de sécurité, de protection des données et de conformité — et plan de remédiation pour une sécurisation de bout en bout.

Version 1.0 · 6 septembre 2026

Périmètre : application Flutter (Android / iOS / Web / Desktop), console d'administration Web, relais Groq (Cloudflare Worker), backend Supabase (PostgreSQL + RLS + Edge Functions), chaîne de paiement CinetPay, CI/CD GitHub Actions.

Méthode : revue de code statique du dépôt `jurisia_app` + analyse d'architecture. Aucun test d'intrusion actif n'a été mené.

CONFIDENTIEL — USAGE INTERNE

Sommaire

1. Résumé exécutif
2. Périmètre, méthode et limites
3. Architecture et surface d'attaque
4. Synthèse des vulnérabilités & matrice de priorités
5. Vulnérabilités critiques
 - JUR-01 — Contournement total du paiement (RPC `jurisia_billing_*`)
6. Vulnérabilités élevées
 - JUR-02 — Quotas d'abonnement appliqués côté client uniquement
 - JUR-03 — Relais Groq : accès anonyme, quota par IP, fail-open
 - JUR-04 — Aucune protection anti-bot à l'inscription
 - JUR-05 — PII sensibles lisibles par tout rôle « staff »
 - JUR-06 — Intégrité du journal d'audit
7. Vulnérabilités moyennes (JUR-07 à JUR-14)
8. Vulnérabilités faibles & hygiène (JUR-15 à JUR-24)
9. Protection des données & conformité (RGPD / APDP Burkina Faso)
10. Plan d'action priorisé (roadmap 30 / 60 / 90 jours)
11. Annexe A — Checklist de configuration Supabase
12. Annexe B — Checklist de durcissement (infra, CI, mobile)
13. Annexe C — Modèle de menace synthétique
14. Annexe D — Points déjà conformes (à préserver)

1. Résumé exécutif

Le socle de sécurité de Jurisia est **correctement pensé dans ses principes** : Row Level Security (RLS) activée sur toutes les tables, secrets serveur (clé Groq, `service_role`) jamais exposés au client, séparation des données utilisateur et des données IA, défense anti-injection de prompt réfléchie, webhook de paiement qui ne fait pas confiance à son propre contenu.

Cependant, la revue identifie **une vulnérabilité critique** et **cinq vulnérabilités élevées** qui, combinées, permettent aujourd'hui à n'importe quel titulaire de compte :

- de **s'octroyer gratuitement le plan payant le plus cher** (Jurisia Pro / Cabinet) de façon indétectable ;
- de **contourner l'ensemble des quotas** du modèle freemium (consultations, mises en relation, Espace pro) ;
- d'**abuser du relais IA** pour faire porter à l'éditeur un coût Groq non maîtrisé ;
- en cas de compromission d'un seul compte du personnel, d'**exfiltrer l'ensemble des demandes de mise en relation** et leur contexte juridique personnel.

La cause racine principale (JUR-01) est un défaut de configuration des droits d'exécution sur les fonctions PostgreSQL : plusieurs fonctions `SECURITY DEFINER` destinées au serveur uniquement restent appelables directement par le client via l'API PostgREST, sans qu'aucune n'ait fait l'objet d'un `REVOKE EXECUTE` .

Action immédiate recommandée (moins d'une heure de travail) : révoquer l'exécution publique de `jurisia_billing_apply` , `jurisia_admin_log` et de toute fonction « serveur uniquement », puis auditer les tables `subscriptions` et `payment_intents` existantes à la recherche d'activations frauduleuses. Voir JUR-01 §5.1.

Répartition des constats

Gravité	Nombre	Identifiants
CRITIQUE	1	JUR-01
ÉLEVÉ	5	JUR-02 à JUR-06
MOYEN	8	JUR-07 à JUR-14
FAIBLE / HYGIÈNE	10	JUR-15 à JUR-24

2. Périmètre, méthode et limites

Composants examinés

Composant	Emplacement	Rôle
Application cliente	<code>lib/</code>	Flutter — Android, iOS, Web, Windows, macOS. Un seul code base.
Console d'administration	<code>lib/admin/</code> , <code>lib/admin_main.dart</code>	Application Flutter Web séparée, déployée sur <code>jurisia-admin.pages.dev</code> .
Relais Groq	<code>server/groq-proxy/src/index.js</code>	Cloudflare Worker — détient la clé Groq, valide, borne, limite le débit.
Base de données	<code>server/supabase/*.sql</code>	PostgreSQL managé (Supabase), 13 migrations, RLS + RPC <code>SECURITY DEFINER</code> .
Edge Functions	<code>supabase/functions/</code>	Deno — <code>billing-checkout</code> , <code>billing-webhook</code> .
Paieement	<code>supabase/functions/_shared/billing.ts</code>	Abstraction <code>BillingProvider</code> — <code>MockProvider</code> + <code>CinetPayProvider</code> (API v1 OAuth).
CI/CD	<code>.github/workflows/ci.yml</code>	Analyse, tests, build, déploiement Cloudflare Pages + Worker.
Suivi d'erreurs	<code>lib/core/monitoring/crash_reporting.dart</code>	Sentry (actif en production).

Méthode

- Revue de code statique manuelle de l'intégralité du code applicatif et serveur du dépôt.
- Analyse des politiques RLS et des fonctions `SECURITY DEFINER` migration par migration.
- Analyse du contrat client ↔ relais Groq et client ↔ Edge Functions.
- Analyse du modèle de menace (abus économique, fuite de données, injection, chaîne d'approvisionnement).

Limites

- **Aucun test d'intrusion actif** n'a été réalisé : les scénarios d'exploitation sont déduits du code et doivent être confirmés en environnement de test.
- La **configuration du tableau de bord Supabase** (confirmation d'e-mail, politique de mot de passe, rate-limit Auth, MFA, région d'hébergement) n'est pas visible dans le dépôt — elle fait l'objet d'une checklist dédiée (Annexe A) à valider manuellement.
- Les **paramètres Cloudflare** (périmètre du token API, WAF, Access) et la **console CinetPay** ne sont pas dans le périmètre du code.
- Revue effectuée à la date indiquée ; tout changement ultérieur du code ou de la configuration invalide potentiellement une partie des constats.

3. Architecture et surface d'attaque

Flux principaux

- **Authentification** : Supabase Auth (e-mail / mot de passe). Le JWT utilisateur est utilisé par l'app, la console admin, le relais Groq et l'Edge Function `billing-checkout`.
- **Données utilisateur** : le client parle directement à PostgREST (`supabase-js`) ; protection = RLS par `auth.uid()`.
- **IA** : client → relais Groq (Cloudflare Worker) → API Groq. Le Worker valide le JWT auprès de Supabase, lit le plan, applique quotas + plafond de tokens.
- **Paiement** : client → `billing-checkout` (JWT) → intention de paiement + redirection CinetPay ; CinetPay → `billing-webhook` (serveur-à-serveur) → re-vérification + activation via `jurisia_billing_apply`.
- **Administration** : console Web → PostgREST / RPC ; protection = présence d'une ligne dans `staff_roles` + gardes internes des fonctions `SECURITY DEFINER`.

Points d'entrée exposés à Internet

Surface	Authentification exigée	Remarque
PostgREST <code>/rest/v1/*</code> (tables)	Selon RLS	Lecture publique volontaire sur <code>plans</code> , <code>ai_limits</code> , <code>legal_documents</code> , <code>legal_articles</code> , <code>ai_prompts</code> (publiés).
PostgREST <code>/rest/v1/rpc/*</code> (fonctions)	Aucune par défaut	Toutes les fonctions <code>public.*</code> sont exécutables par <code>anon</code> / <code>authenticated</code> tant qu'aucun <code>REVOKE</code> n'est posé — cause de JUR-01.
Relais Groq <code>/v1/chat/completions</code>	Non (repli anonyme)	Quota par IP si pas de JWT — voir JUR-03.
<code>billing-checkout</code>	Oui (<code>verify_jwt = true</code>)	CORS <code>*</code> mais pas de cookie → pas de CSRF.
<code>billing-webhook</code>	Non (<code>verify_jwt = false</code>)	Sécurité = re-vérification authentifiée auprès de CinetPay + idempotence.
Console admin (SPA)	Oui (session + rôle)	SPA servie publiquement ; aucune couche réseau devant — voir JUR-08.
App Web (SPA)	Non	Aucune CSP, aucun en-tête de sécurité — voir JUR-16.

Valeurs publiques assumées, correctement classées comme non secrètes : URL et `anon key` Supabase, URL du relais Groq, DSN Sentry (écriture seule), `account_id` Cloudflare. Cette posture reste valable à condition que JUR-01 soit corrigé — sinon l' `anon key` devient de fait une clé d'accès aux abonnements gratuits.

4. Synthèse des vulnérabilités & matrice de priorités

ID	Titre	Gravité	Effort	Priorité
JUR-01	Contournement total du paiement via RPC <code>jurisia_billing_apply</code>	CRITIQUE	Faible	P0 — immédiat
JUR-02	Quotas d'abonnement appliqués côté client uniquement	ÉLEVÉ	Moyen	P1
JUR-03	Relais Groq : accès anonyme, quota par IP, fail-open KV	ÉLEVÉ	Moyen	P1
JUR-04	Aucune protection anti-bot à l'inscription	ÉLEVÉ	Faible	P1
JUR-05	PII sensibles lisibles par tout rôle « staff »	ÉLEVÉ	Moyen	P1
JUR-06	Intégrité du journal d'audit (<code>jurisia_admin_log</code> ouvert au staff)	ÉLEVÉ	Faible	P1
JUR-07	Configuration Supabase Auth non durcie (à auditer)	MOYEN	Faible	P1
JUR-08	Console d'administration sans couche réseau / MFA	MOYEN	Faible	P2
JUR-09	Identité staff mise en cache toute la session	MOYEN	Faible	P2
JUR-10	Studio de prompts : risque d'initié	MOYEN	Faible	P2
JUR-11	Réinjection de contexte dérivé utilisateur dans le prompt système	MOYEN	Faible	P2
JUR-12	Métadonnées d'inscription non validées côté serveur	MOYEN	Faible	P2
JUR-13	CORS * sur les Edge Functions	MOYEN	Faible	P2
JUR-14	Relais Groq : passthrough intégral des en-têtes de réponse	MOYEN	Faible	P2
JUR-15	Journalisation de corps de réponse (<code>print</code>)	FAIBLE	Faible	P3
JUR-16	Absence de Content-Security-Policy / en-têtes de sécurité Web	FAIBLE	Faible	P2
JUR-17	<code>.wrangler/cache/pages.json</code> versionné	FAIBLE	Faible	P3
JUR-18	Keystore Android en copie unique	FAIBLE	Faible	P2
JUR-19	Dépendance runtime <code>esm.sh</code> dans les Edge Functions	FAIBLE	Faible	P3
JUR-20	Périmètre du token Cloudflare CI à minimiser	FAIBLE	Faible	P2
JUR-21	Historique de consultations en clair dans le <code>localStorage</code> (Web)	FAIBLE	Faible	P3
JUR-22	Pas de parcours « supprimer mon compte / mes données »	FAIBLE	Moyen	P2 (conformité)
JUR-23	Absence de politique de rétention / purge	FAIBLE	Moyen	P3
JUR-24	Sous-traitant IA (Groq, hors zone) non explicité aux utilisateurs	FAIBLE	Faible	P2 (conformité)

Effort : Faible < 1 j · Moyen 1–5 j · Élevé > 5 j. Priorité : P0 immédiat, P1 sous 30 j, P2 sous 60 j, P3 sous 90 j.

5. Vulnérabilités critiques

JUR-01 — Contournement total du paiement via les RPC `jurisia_billing_*`

Gravité : **CRITIQUE** | Composant : Base de données / RPC | Fichiers : `server/supabase/migration_009_billing.sql`, ensemble des migrations 006-013

Description

La fonction `public.jurisia_billing_apply(p_transaction_id, p_new_status, p_period_end)` est déclarée `SECURITY DEFINER` et ne contient aucune vérification d'appelant : ni `auth.uid()`, ni `jurisia_is_staff()`, ni secret partagé. Elle est censée n'être invoquée que par l'Edge Function `billing-webhook` avec la clé `service_role`.

Or aucune migration ne pose de `REVOKE EXECUTE ... FROM anon, authenticated` (vérifié : aucune occurrence de `revoke` dans `server/supabase/`). Le comportement par défaut de Supabase accorde `EXECUTE` sur toute fonction du schéma `public` aux rôles `anon` et `authenticated`, exposés via PostgREST à l'adresse `POST /rest/v1/rpc/<fonction>`. Le fait que les fonctions d'administration renvoient une erreur métier (« réservé au personnel ») plutôt qu'un refus d'accès confirme que ces RPC sont atteignables par un appelant non privilégié, et que seules leurs gardes internes les protègent — garde que `jurisia_billing_apply` n'a pas.

Scénario d'exploitation

- Un utilisateur authentifié appelle `POST /rest/v1/rpc/jurisia_billing_create_intent` avec `{ p_plan_code: "pro", p_transaction_id: "<valeur qu'il choisit>", p_provider: "mock" }`. La fonction vérifie `auth.uid()`, lit le tarif dans `plans`, et crée une intention `pending` rattachée à son propre compte.
- Il appelle ensuite `POST /rest/v1/rpc/jurisia_billing_apply` avec `{ p_transaction_id: "<même valeur>", p_new_status: "paid" }`. Sans aucune vérification, la fonction passe l'intention à `paid`, insère une ligne `subscriptions active` (plan Pro / Cabinet / +), et écrit au journal d'audit une entrée « paiement confirmé (mock) ».

Impact

- N'importe quel titulaire de compte obtient **gratuitement le plan le plus cher**, immédiatement, sans transaction réelle.
- L'attaque est **indétectable par les moyens actuels** : le journal d'audit affiche un « paiement confirmé » standard.
- Perte de revenu directe + fausse comptabilité d'abonnements.
- Vecteur secondaire : un attaquant qui connaît le `transaction_id` d'un paiement `pending` légitime peut le forcer.

Remédiation

5.1 — Correctif immédiat (P0)

```
-- Révoquer l'exécution publique de toute fonction "serveur uniquement".
revoke execute on function public.jurisia_billing_apply(text, text, timestampz) from anon, authenticated;
revoke execute on function public.jurisia_admin_log(text, text, text, jsonb, jsonb, text) from anon, authenticated;

-- Garde interne défensive : une vraie exécution webhook n'a pas de contexte utilisateur.
create or replace function public.jurisia_billing_apply(...)
returns void as $$
begin
  if auth.uid() is not null then
    raise exception 'appel interdit depuis un contexte utilisateur';
  end if;
  -- ... corps existant ...
end;
$$ language plpgsql volatile security definer set search_path = public;
```

5.2 — Correctif structurel (P1) — approche « deny by default »

```
-- 1. Retirer l'exécution par défaut sur TOUT le schéma public.
alter default privileges in schema public revoke execute on functions from anon, authenticated;
revoke execute on all functions in schema public from anon, authenticated;

-- 2. Ré-accorder explicitement, fonction par fonction, UNIQUEMENT celles
-- réellement destinées au client :
grant execute on function public.jurisia_entitlements() to authenticated;
grant execute on function public.jurisia_plan_code() to authenticated, anon;
grant execute on function public.jurisia_record_usage(text) to authenticated;
grant execute on function public.jurisia_billing_create_intent(text, text, text) to authenticated;
grant execute on function public.jurisia_admin_list_staff() to authenticated;
-- ... (chaque jurisia_admin_* appelée par la console) ...
-- jurisia_billing_apply : n'est JAMAIS ré-accordée (service_role uniquement).
```

Vérification post-incident : exécuter en base `select * from public.subscriptions where psp = 'mock' or psp is null;` et `select * from public.payment_intents where transaction_id !~ '^jurisia-[0-9]+-[0-9a-f]{8}$';` (le format légitime est généré par `newTransactionId()`). Toute ligne suspecte = activation frauduleuse à révoquer.

5.3 — Défense en profondeur

- Ajouter à `billing-webhook` une vérification de la signature / `x-token` CinetPay lorsque l'API v1 le permet (aujourd'hui compensé par la re-vérification `GET /v1/payment/{id}`).
- Alerte automatique (Sentry / cron) sur toute activation d'abonnement dont le `psp` n'est pas `cinetpay` en production.

- Test d'intégration en CI qui échoue si une RPC « serveur uniquement » redevient exécutable par `authenticated` .

6. Vulnérabilités élevées

JUR-02 — Les quotas d'abonnement ne sont appliqués que côté client

Gravité : **ÉLEVÉ** | Composant : Application cliente / logique métier | Fichiers : `lib/core/entitlements/entitlements_controller.dart`, `lib/core/entitlements/plan.dart`

Description

`EntitlementsController.tryConsume()` lit et écrit un compteur dans `LocalCache` (localStorage sur le Web) et décide **localement** si l'action est permise. L'appel serveur `jurisia_record_usage` est `unawaited` et n'a aucun pouvoir de blocage. La réconciliation serveur applique un `max()` : elle ne peut que remonter le compteur, jamais refuser une action.

Impact

- Un client modifié — ou un simple contournement de la vérification locale — annule « 3 consultations/mois », « 1 mise en relation/mois », le mode approfondi, l'accès Espace pro, le coffre-fort.
- Vider le `localStorage` réinitialise les compteurs ; bloquer le RPC `jurisia_entitlements` au niveau réseau empêche toute réconciliation.
- Le seul rempart serveur réel est la limite de débit du Worker Groq — qui protège le coût Groq, **pas** le modèle économique (fonctionnalités non-IA, volume mensuel).

Remédiation

- Créer une RPC serveur `jurisia_try_consume(p_feature text) returns boolean`, `SECURITY DEFINER`, qui : lit le plan réel dans `subscriptions`, lit le quota du plan, lit `usage_counters`, et retourne `false` (ou lève une exception) si le quota est atteint, sinon incrémente atomiquement et retourne `true`.
- Le client n'a plus accès à l'incrément libre — seulement à cette fonction qui fait autorité.
- Pour les capacités payantes qui produisent une écriture (documents Espace pro, export PDF), gater aussi la table concernée par une politique RLS conditionnée au plan, ou par une RPC dédiée.
- Conserver le repli gracieux : si le serveur ne répond pas, autoriser au quota Découverte (jamais « ouvrir les vannes »).

JUR-03 — Relais Groq : accès anonyme, quota par IP, fail-open

Gravité : **ÉLEVÉ** | Composant : Cloudflare Worker | Fichier : `server/groq-proxy/src/index.js`

Description & impact

- Aucun jeton exigé.** Un appel sans en-tête `Origin` (curl, script, application native) et sans `Authorization` est traité comme « anonyme » et accepté, avec `ANON_LIMITS` = 20 req/min, 300 req/j **par adresse IP**. Un pool d'adresses (botnet, proxies résidentiels) permet de consommer du crédit Groq sans limite pratique — la facture Groq est celle de l'éditeur.
- Fail-open.** `checkRateLimit()` retourne `{ allowed: true }` si `RATE_LIMIT_KV` n'est pas lié. Un binding KV cassé ou un déploiement incomplet supprime toute limite silencieusement.
- Comptage non atomique** (documenté dans le code) : des rafales concurrentes dépassent le quota affiché.
- Multiplication par compte.** Chaque compte créé donne des limites supérieures aux limites anonymes (`decouverte` : 150/j, `plus` : 600/j...). Couplé à JUR-04, une ferme d'inscriptions multiplie le quota.

Remédiation

- Exiger un JWT Supabase valide pour tout appel non-`localhost` ; supprimer le repli anonyme (ou le plafonner à un quota **global** très bas, pas par IP).
- Fail-closed si `RATE_LIMIT_KV` est absent (au minimum : refuser + émettre une alerte).
- Migrer le comptage vers un **Durable Object** ou le binding natif **Cloudflare Rate Limiting** pour un décompte exact.
- Poser un plafond de dépense côté Groq (hard limit / alerte de facturation) et une alerte Workers Logs sur les pics de volume.
- Activer le WAF Cloudflare (règles de challenge sur les motifs d'abus, bot fight mode).

JUR-04 — Aucune protection anti-bot à l'inscription

Gravité : **ÉLEVÉ** | Composant : Supabase Auth / écran d'inscription | Fichiers : `lib/features/auth/`, configuration Supabase Auth

Description & impact

Inscription e-mail / mot de passe sans CAPTCHA, mot de passe minimum 6 caractères imposé côté client seulement, pas de limitation applicative visible. Couplé à JUR-01 (chaque compte = paiement contournable) et JUR-03 (chaque compte = quota IA supérieur), l'ouverture de comptes en masse devient un vecteur d'abus direct et un vecteur de bruit sur la base d'utilisateurs.

Remédiation

- Activer la protection CAPTCHA (hCaptcha ou Cloudflare Turnstile) dans Supabase Auth.
- Rendre la **confirmation d'e-mail obligatoire** (voir JUR-07).
- Politique de mot de passe : minimum 10–12 caractères, activer la « leaked password protection » Supabase (vérification HavelBeenPwned).
- Rate-limit d'inscription et de connexion par IP (paramètre Supabase Auth + WAF).

JUR-05 — Données personnelles sensibles lisibles par tout rôle « staff »

Gravité : **ÉLEVÉ** | Composant : RLS / RPC d'administration | Fichiers : `migration_008`, `migration_011`, `migration_012`, `migration_013`

Description

Toutes les politiques transverses et toutes les RPC de lecture de la console utilisent `jurisia_is_staff()` — c'est-à-dire **n'importe quel rôle**, y compris `analyst`, `support_agent`, `partner_manager`. Les données concernées sont pourtant très sensibles :

- `professional_contact_requests` : nom, coordonnées (`contact_info`), et `message` décrivant une situation juridique personnelle.
- `subscriptions`, `usage_counters` : qui paie quoi, intensité d'usage.
- `jurisia_admin_list_staff` : e-mails de tout le personnel et graphe « qui a accordé quel rôle ».
- `jurisia_admin_list_document_drafts` / `list_prompts` : tout le contenu en préparation.

Point positif : les conversations Litige (`litigation_messages`) et l'Espace pro (`professional_drafting_results`) ne sont *pas* exposés au staff.

Impact

La compromission d'un **seul** compte du personnel (hameçonnage, réutilisation de mot de passe, poste non verrouillé) entraîne la fuite de l'ensemble des demandes de mise en relation et de leur contexte juridique.

Remédiation

- Politiques et RPC **par rôle** : `jurisia_has_role('support_agent')` pour les seules demandes de contact, `analyst` restreint à des vues agrégées **sans PII**, etc.
- Masquer `contact_info` / `message` sauf pour les rôles qui en ont l'usage opérationnel.
- Rendre la **MFA obligatoire** pour tout compte disposant d'un rôle dans `staff_roles`.
- Journaliser chaque lecture de demande de contact (accès traçable).

JUR-06 — Intégrité du journal d'audit

Gravité : **ÉLEVÉ** | Composant : RPC d'administration | Fichier : `server/supabase/migration_008_admin_operations.sql`

Description & impact

`jurisia_admin_log(p_action, p_target_type, ...)` n'exige que `jurisia_is_staff()`. Tout membre du personnel peut donc **écrire des entrées d'audit arbitraires** : fausses actions, brouillage pour noyer une action réelle, attribution trompeuse. Le journal est présenté comme « ajout seul, jamais modifié » — mais son *contenu* n'est pas fiable, ce qui ruine sa valeur probante en cas d'incident.

Remédiation

- `revoke execute on function public.jurisia_admin_log(...) from anon, authenticated;` — la fonction ne doit être appelée qu'en interne via `perform` depuis les autres fonctions `SECURITY DEFINER`.
- Toujours forcer `actor_id := auth.uid()` (déjà le cas) et interdire tout `actor_id` passé en paramètre.
- Envisager une répliquation du journal vers un stockage en écriture seule hors base (bucket append-only, ou export Sentry / SIEM).

7. Vulnérabilités moyennes

JUR-07 — Configuration Supabase Auth non durcie (à auditer)

Composant : Tableau de bord Supabase

Éléments non visibles dans le code, à contrôler explicitement (voir Annexe A) :

- Confirmation d'e-mail obligatoire — sinon inscription avec l'e-mail d'autrui et session immédiate.
- Durée de vie du JWT, rotation des refresh tokens.
- Rate-limit Auth (protection contre le bruteforce sur `signInWithPassword`).
- Longueur minimale de mot de passe (le client impose 6, très faible) + leaked password protection.
- Schémas exposés par PostgreSQL limités à `public` ; `auth` et `storage` non exposés.
- URL de redirection Auth restreintes aux domaines JurisIA.
- Région d'hébergement du projet et implications RGPD / APDP (voir §9).

JUR-08 — Console d'administration sans couche réseau ni MFA

Fichier : `lib/admin/` , déploiement Cloudflare Pages `jurisia-admin`

Le SPA d'administration est servi publiquement ; la sécurité repose entièrement sur `staff_roles` et les gardes RPC. Aucune défense en profondeur : pas de Cloudflare Access / SSO, pas d'IP allowlist, pas de MFA. Même projet Supabase, même `anon key` , même origine d'authentification que l'app grand public.

Remédiation

- Placer **Cloudflare Access** (politique par e-mail + SSO + MFA) devant `jurisia-admin.pages.dev` .
- MFA Supabase obligatoire sur les comptes staff.
- À terme : domaine d'authentification distinct pour la console.

JUR-09 — Identité staff mise en cache toute la session

Fichier : `lib/admin/auth/admin_auth_gate.dart`

`currentIdentity()` est appelé une seule fois dans `initState` . Une révocation de rôle (`jurisia_admin_revoke_staff_role`) ne prend effet dans l'UI qu'au prochain rechargement complet de l'onglet — fenêtre d'accès résiduel. Les RPC restent protégées côté serveur (le risque est limité à l'affichage).

Remédiation

Re-vérifier l'identité périodiquement (ex. toutes les 5 min et à chaque navigation sensible) ; réduire la durée de vie de session de la console.

JUR-10 — Studio de prompts : risque d'initié

Fichier : `server/supabase/migration_013_ai_prompts.sql` , `lib/core/ai/prompt_overrides.dart`

Un `content_editor` rédige un addendum de prompt, le teste, puis un `super_admin` le publie. **Bonne mitigation existante :** l'addendum est *ajouté* au prompt fixe, jamais substitué, et le protocole de sortie JSON caché est préservé. Restent : un addendum malveillant peut orienter l'IA vers des conseils juridiques erronés pour tous les utilisateurs simultanément, ou tenter de faire divulguer le bloc caché.

Remédiation

Revue de *diff* obligatoire à la publication ; conservation de l'historique (déjà : `archived`) avec rollback en un clic ; test automatisé anti-régression du format de sortie sur chaque prompt candidat.

JUR-11 — Réinjection de contexte dérivé utilisateur dans le prompt système

Fichier : `lib/features/litigation/data/datasources/litigation_system_prompt.dart`

`withContext()` réinjecte `grid.faits` , `grid.forces` , etc. (contenu produit par le modèle à partir des messages utilisateur) dans le **prompt système** du tour suivant. C'est la seule entorse à la règle « jamais de contenu dérivé de l'utilisateur dans le système ». Impact faible (le contenu a déjà transité par le modèle), mais à corriger par principe.

Remédiation

Réinjecter ce rappel comme message `assistant` (ou `user` « note interne ») plutôt que dans le bloc système. La défense anti-injection de l'Espace pro (`professional_system_prompt.dart`) est, elle, exemplaire — même modèle à généraliser.

JUR-12 — Métadonnées d'inscription non validées côté serveur

Fichier : `server/supabase/schema.sql` (`handle_new_user`)

`full_name` et `profession` proviennent de `raw_user_meta_data` (fixé par le client), sans contrainte de longueur ni de valeur côté serveur (`AppInputLimits` est purement client). Pas de risque XSS (Flutter n'interprète pas le HTML), mais stockage de chaînes arbitraires affichées dans la console admin.

Remédiation

```
alter table public.profiles
  add constraint profiles_full_name_len check (char_length(full_name) <= 120),
  add constraint profiles_profession_valid
    check (profession is null or profession in ('avocat','notaire','juriste','huissier','greffier','juge','etudiant','particulier'));
```

Valider aussi dans `handle_new_user` (tronquer / rejeter).

JUR-13 — CORS * sur les Edge Functions

Fichier : `supabase/functions/_shared/cors.ts`

`Access-Control-Allow-Origin: *`. `billing-checkout` exige un `Authorization: Bearer` (pas de cookie) → pas de CSRF exploitable, impact faible. À restreindre malgré tout aux origines JurisIA, par cohérence avec le Worker Groq.

JUR-14 — Relais Groq : passthrough intégral des en-têtes de réponse

Fichier : `server/groq-proxy/src/index.js`

`new Headers(groqResponse.headers)` recopie **tous** les en-têtes renvoyés par Groq vers le client — ce qui peut divulguer des en-têtes internes (quotas d'organisation, identifiants de requête, informations d'infrastructure).

Remédiation

Recopier une liste blanche stricte (`content-type` , `cache-control`) et construire le reste des en-têtes soi-même.

8. Vulnérabilités faibles & hygiène

ID	Constat	Remédiation
JUR-15	<code>lib/core/ai/groq_api_datasource.dart</code> : <code>print('Groq API error ... body: \$body')</code> écrit le corps de réponse (extraits de texte du modèle possibles) dans la console navigateur.	Retirer ou conditionner à <code>kDebugMode</code> ; ne jamais journaliser de contenu utilisateur/modèle en release.
JUR-16	Aucune Content-Security-Policy ni en-tête de sécurité sur l'app Web ni sur la console (<code>web/index.html</code> n'en pose aucun ; Cloudflare Pages n'ajoute rien par défaut). Un XSS éventuel s'exécute sans contrainte ; la console admin est <i>iframeable</i> (clickjacking).	Ajouter un fichier <code>_headers</code> (Pages) : <code>Content-Security-Policy</code> avec <code>connect-src</code> limité à Supabase + Worker Groq + Sentry, <code>frame-ancestors 'none'</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , <code>Permissions-Policy</code> minimal.
JUR-17	<code>.wrangler/cache/pages.json</code> est versionné (apparaît en modifié). Contenu non secret (<code>account_id</code> , nom de projet) mais c'est de l'état local de build.	Ajouter <code>.wrangler/</code> au <code>.gitignore</code> racine ; <code>git rm --cached .</code>
JUR-18	Keystore de signature Android (<code>android/key.properties</code> , <code>*.jks</code>) en copie unique sur le poste du porteur. Perte = impossibilité de publier une mise à jour sous la même identité Play Store.	Sauvegarde chiffrée hors dépôt (gestionnaire de secrets / coffre) ; documenter la procédure de restauration.
JUR-19	<code>supabase/functions/_shared/supabase.ts</code> importe <code>@supabase/supabase-js</code> depuis <code>esm.sh</code> (épinglé @2.47.10). Tiers dans la chaîne d'exécution des Edge Functions.	Passer à un spécifier <code>npm</code> : (supporté par Deno / Supabase) ou verrouiller par <code>deno.lock</code> + <code>import map</code> .
JUR-20	Le secret CI <code>CLOUDFLARE_API_TOKEN</code> déploie Pages + Workers. Un token trop large en cas de fuite du dépôt / d'un runner compromis.	Créer le token avec le périmètre minimal (Pages:Edit + Workers Scripts:Edit sur le compte concerné uniquement) ; rotation planifiée ; environnements GitHub protégés (approbation avant déploiement <code>main</code>).
JUR-21	<code>lib/core/storage/local_cache.dart</code> : sur le Web, les titres de consultations (susceptibles de résumer un litige) et le plan sont en clair dans le <code>localStorage</code> . Lisible sur poste partagé et par tout script de même origine.	Acceptable pour des titres ; ne jamais étendre au contenu des messages. Purger le cache à la déconnexion. Envisager un avertissement « appareil partagé ».
JUR-22	Aucun parcours utilisateur « supprimer mon compte / mes données ». Les <code>on delete cascade</code> sont en place mais rien ne déclenche la suppression de l'utilisateur <code>auth.users</code> .	RPC <code>jurisia_delete_my_account()</code> (ou Edge Function <code>service_role</code>) + écran dédié ; journaliser la demande.
JUR-23	<code>usage_events</code> , <code>admin_audit_log</code> , <code>litigation_*</code> croissent sans purge ni anonymisation.	Politique de rétention documentée (ex. événements > 24 mois anonymisés ; conversations inactives > N mois archivées/supprimées) ; tâche planifiée <code>pg_cron</code> .
JUR-24	Le contenu des consultations transite par Groq (hors zone Burkina/UE). Non explicité dans la politique de confidentialité examinée.	Mentionner Groq comme sous-traitant, la localisation du traitement, et la politique de non-entraînement de l'API dans la politique de confidentialité et les CGU.

9. Protection des données & conformité (RGPD / APDP Burkina Faso)

JurisIA traite des données de catégorie sensible : problèmes juridiques personnels détaillés (Litige), contrats d'affaires (Espace pro), coordonnées et situation (mise en relation). La loi burkinabè n° 001-2021 portant protection des données à caractère personnel (APDP) et, pour les utilisateurs concernés, le RGPD, s'appliquent.

Exigence	État actuel	Action
Information des personnes (finalités, base légale, durées)	CGU et politique de confidentialité présentes (<code>lib/core/legal/</code>)	Vérifier l'exhaustivité : finalités par module, base légale, durées de conservation, liste des sous-traitants (Supabase, Cloudflare, Groq, Sentry, CinetPay), transferts hors zone.
Consentement / acceptation	Acceptation CGU horodatée à l'inscription (<code>terms_accepted_at</code>)	Conserver la version des CGU acceptée (pas seulement la date).
Droit d'accès / portabilité	Absent	Export des données de l'utilisateur (JSON) à la demande.
Droit à l'effacement	Absent (JUR-22)	Parcours de suppression de compte + purge en cascade + confirmation.
Minimisation	Partiellement respectée	Revoir les lectures staff transverses (JUR-05) ; ne collecter <code>contact_info</code> que sous une forme nécessaire.
Sécurité (art. 32 RGPD)	RLS + chiffrement au repos Supabase par défaut	Corriger JUR-01 à JUR-06 ; envisager un chiffrement applicatif des colonnes les plus sensibles (<code>professional_contact_requests.message</code> , <code>contact_info</code>).
Transferts hors zone	Groq (USA), Cloudflare, Supabase (région à confirmer)	Documenter ; choisir une région Supabase UE si possible ; encadrer contractuellement.
Registre des traitements / DPIA	Non constaté	Tenir un registre ; une analyse d'impact (DPIA/AIPD) est recommandée vu la sensibilité juridique.
Notification de violation	Non constaté	Procédure interne : détection (Sentry + alertes DB), qualification, notification APDP / personnes sous 72 h.
Responsabilité du conseil IA	Avertissement présent (<code>ai_disclaimer_screen.dart</code>)	Le maintenir visible avant chaque usage à enjeu ; formulation « aide à la compréhension, pas un avis juridique engageant ».

10. Plan d'action priorisé

P0 — Immédiat (cette semaine)

1. **JUR-01** — `REVOKE EXECUTE` sur `jurisia_billing_apply` et `jurisia_admin_log` ; garde interne anti-contexte-utilisateur sur `jurisia_billing_apply`.
2. **JUR-01** — Audit des `subscriptions` / `payment_intents` existants ; révocation des activations frauduleuses.
3. Alerte de surveillance sur les activations d'abonnement non-CinetPay en production.

P1 — Sous 30 jours

4. **JUR-01** — Migration « deny by default » sur toutes les fonctions `public.*` + ré-attribution explicite (§5.2). Test CI de non-régression.
5. **JUR-02** — RPC serveur `jurisia_try_consume` faisant autorité sur les quotas ; câblage client.
6. **JUR-03** — Relais Groq : JWT exigé, fail-closed KV, liste blanche d'en-têtes, plafond de dépense Groq, WAF.
7. **JUR-04** — CAPTCHA + confirmation e-mail + politique de mot de passe + rate-limit Auth.
8. **JUR-05** — Politiques RLS / RPC par rôle staff ; MFA staff obligatoire.
9. **JUR-06** — `jurisia_admin_log` en appel interne uniquement.
10. **JUR-07** — Revue complète de la configuration Supabase (Annexe A).

P2 — Sous 60 jours

11. **JUR-08** — Cloudflare Access devant la console admin.
12. **JUR-16** — CSP + en-têtes de sécurité (app & admin).
13. **JUR-09 / 10 / 11 / 12 / 13 / 14** — corrections ciblées.
14. **JUR-18** — Sauvegarde chiffrée du keystore Android.
15. **JUR-20** — Réduction du périmètre du token Cloudflare CI + environnements GitHub protégés.
16. **JUR-22 / 24** — Parcours de suppression de compte ; mise à jour de la politique de confidentialité (sous-traitants, transferts).

P3 — Sous 90 jours

17. **JUR-15 / 17 / 19 / 21 / 23** — hygiène.
18. Mise en place d'un registre des traitements + DPIA.
19. Test d'intrusion externe (idéalement après P0/P1) sur l'API PostgREST, le relais Groq, la chaîne de paiement et la console admin.
20. Revue de sécurité récurrente (à chaque nouvelle migration / nouvelle Edge Function) + intégration d'un scan de dépendances (`flutter pub outdated` , `npm audit` , Dependabot).

Annexe A — Checklist de configuration Supabase

Point	Cible	OK
Confirmation d'e-mail	Obligatoire avant première session	☐
CAPTCHA (Auth)	Activé (Turnstile / hCaptcha)	☐
Longueur min. mot de passe	≥ 10–12	☐
Leaked password protection	Activée	☐
Rate limits Auth (sign-in / sign-up / OTP)	Valeurs resserrées	☐
MFA	Disponible ; obligatoire pour le staff	☐
Durée JWT / rotation refresh token	JWT ≤ 1 h ; rotation activée ; réutilisation détectée	☐
Schémas exposés PostgREST	<code>public</code> uniquement	☐
<code>EXECUTE</code> par défaut sur <code>public</code>	Révoqué pour <code>anon</code> / <code>authenticated</code> ; ré-accordé explicitement (JUR-01)	☐
Politiques RLS	Revue par rôle ; pas de <code>using (true)</code> hors lecture publique volontaire	☐
URL de redirection Auth	Liste blanche des domaines JurisIA	☐
Région d'hébergement	Documentée ; UE si possible	☐
Sauvegardes PITR	Activées ; test de restauration effectué	☐
Secrets Edge Functions	<code>SERVICE_ROLE_KEY</code> , clés CinetPay — jamais hors des fonctions	☐
Logs & alertes DB	Alertes sur pics d'erreurs / activations anormales	☐

Annexe B — Checklist de durcissement (infra, CI, mobile)

Domaine	Point	OK
Cloudflare	WAF / Bot Fight Mode actifs sur les zones JurisIA	☐
Cloudflare	Access (SSO + MFA) devant <code>jurisia-admin.pages.dev</code>	☐
Cloudflare	Token API CI au périmètre minimal + rotation	☐
Worker Groq	JWT exigé, fail-closed KV, plafond de dépense Groq	☐
Web	Fichier <code>_headers</code> : CSP, <code>frame-ancestors 'none'</code> , nosniff, Referrer-Policy	☐
CI/CD	Environnements GitHub protégés (approbation avant deploy <code>main</code>)	☐
CI/CD	Scan de dépendances (Dependabot / <code>npm audit</code> / <code>pub outdated</code>)	☐
CI/CD	Scan de secrets (gitleaks) en pre-commit et en CI	☐
Android	Keystore sauvegardé chiffré hors dépôt	☐
Android/iOS	Certificate pinning envisagé pour Supabase / Worker	☐
Mobile	Pas de secret dans le bundle (vérifié : clé Groq absente — OK)	☒
Sentry	<code>sendDefaultPii=false</code> , pas de screenshot, pas de breadcrumb HTTP (vérifié — OK)	☒

Annexe C — Modèle de menace synthétique

Acteur	Objectif	Vecteurs	Constats liés
Utilisateur opportuniste	Accès payant gratuit, quotas illimités	Appels RPC directs, client modifié	JUR-01, JUR-02
Attaquant économique	Faire exploser la facture Groq	Relais anonyme, ferme de comptes, botnet	JUR-03, JUR-04
Attaquant opportuniste (données)	Exfiltrer des PII juridiques	Compromission d'un compte staff, phishing	JUR-05, JUR-06, JUR-08
Initié (personnel)	Abus de privilège, sabotage de contenu	Rôle staff légitime, Studio de prompts, journal d'audit	JUR-05, JUR-06, JUR-10

Attaquant Web	Vol de session, clickjacking	XSS, absence de CSP, iframe de la console	JUR-16, JUR-21
Chaîne d'approvisionnement	Exécution de code via dépendance	<code>esm.sh</code> , token CI, actions GitHub	JUR-19, JUR-20
Manipulation de l'IA	Détourner la tâche, exfiltrer le contexte	Injection de prompt, faux bloc caché	JUR-11 (résiduel — bien traité)

Annexe D — Points déjà conformes (à préserver)

- RLS activée sur **toutes** les tables, avec politiques « propriétaire » cohérentes (`auth.uid() = user_id`).
- Clé Groq et `service_role` jamais exposées au client ; le client ne manipule aucun secret.
- Séparation nette données utilisateur / données IA : les conversations Litige et l'Espace pro ne sont **pas** lisibles par le personnel.
- Défense anti-injection de prompt explicite et bien construite dans l'Espace pro (contenu utilisateur toujours en `role: user`, jamais dans le système).
- Parsing du bloc caché côté client avec replis sûrs (`FormatException`, `TypeError`, `orElse`).
- `search_path` figé (`set search_path = public`) sur toutes les fonctions `SECURITY DEFINER`.
- `billing-webhook` ne fait jamais confiance à son propre corps : re-vérification authentifiée auprès de CinetPay, `transaction_id` généré côté serveur, `jurisia_billing_apply` idempotent, garde anti-altération de montant.
- Modèle Groq unique autorisé + plafond `max_tokens` borné côté serveur (contrôle de coût).
- Sentry configuré sans PII (pas de corps de requête, pas de capture d'écran, pas de session replay).
- Addendum de Studio de prompts qui **n'écrase jamais** le prompt fixe ; double rôle édition / publication ; test obligatoire avant publication.
- CORS du relais Groq restreint aux domaines JurisIA (contrairement à un `*`).
- Secrets locaux correctement exclus du dépôt (`.env`, `key.properties`, `*.jks`, `.dev.vars`).

Fin du dossier. — Ce document reflète l'état du code à la date indiquée. À réviser après chaque lot de correctifs et à l'ajout de toute nouvelle migration ou Edge Function.